Assignment 3: Transformer is All You Need

A Tiny Causal Transformer Trained on Shakespeare

Ayush Verma

COMS4995 Applied Machine Learning, Spring 2026

GitHub Repo: https://github.com/LogicalVerse/applied-machine-learning-assignment-3

1 Introduction

For this assignment I built a small causal Transformer language model from scratch in PyTorch and trained it on the Tiny Shakespeare corpus for next-token prediction. I wanted this project to be more than a training exercise. My main goal was to understand what each part of the model was doing, why those parts are arranged the way they are, and what the model actually learns once training is finished. Because of that, I paid as much attention to the analysis as I did to the final perplexity. I kept the model intentionally small so that I could inspect it closely. A tiny model trains quickly, but it also makes it much easier to look at attention patterns, compare heads, and run short ablations without turning the whole assignment into a compute problem. In this report I describe the data pipeline, the architecture, the training setup, the main attention plots, a few extra diagnostics that helped me understand the model better, and a short ablation study at the end.

2 Data Pipeline

2.1 Corpus and Tokenizer

I used the Tiny Shakespeare corpus, which is about 1.1 MB of raw text and roughly 40,000 lines of dialogue. Since the assignment asked for subword tokenization, I trained a byte-level BPE tokenizer with a vocabulary cap of 500 tokens. I chose byte-level pre-tokenization because it guarantees that every character in the corpus can be represented, even if the tokenizer does not learn a large merge for it. Encoding the full corpus produced 582,954 tokens, which gives a compression ratio of 1.91 characters per token. That felt like a good balance for this setting: much more compact than character-level tokenization, but still small enough to keep the model lightweight.

Figure 1 shows the token statistics. The left panel follows the long-tail shape I expected: a few tokens appear extremely often, while most tokens are much rarer. The right panel is also reassuring. The most common pieces are spaces, punctuation marks, short letters, and short word fragments such as "·the" and "·to." That is exactly what I would want a small subword tokenizer to learn on this corpus. If the frequency curve had looked unusually flat, or if the top tokens were full of strange merges, I would have treated that as a sign that the tokenizer was not behaving well.

2.2 Sequences and Splits

After tokenization, I built overlapping windows of length 51 with a stride of 8. For each window, the first 50 tokens become the input and the same window shifted by one token becomes the target. This is the standard next-token prediction setup: at every position the model tries to predict the following token using only the current and previous positions. With this choice of stride, the full corpus produced 72,863 examples in total. I then split those windows into 58,290 training examples and 14,573 validation examples, which is an 80/20 split with no test set, matching the assignment specification. I used a stride of 8 rather than 1 because a stride of 1 would have produced many near-duplicate windows and would have made the dataset much larger than I needed for this experiment. The smaller stride still keeps plenty of overlap, but it makes training faster and easier to manage.

3 Model Architecture

The final model is a small pre-norm causal Transformer with the following structure:

- token embedding of size 128,
- fixed sinusoidal positional encoding added to the token embedding,
- two Transformer blocks,
- four attention heads per block with head dimension 32,
- feed-forward network $128 \to 256 \to 128$ with GELU,

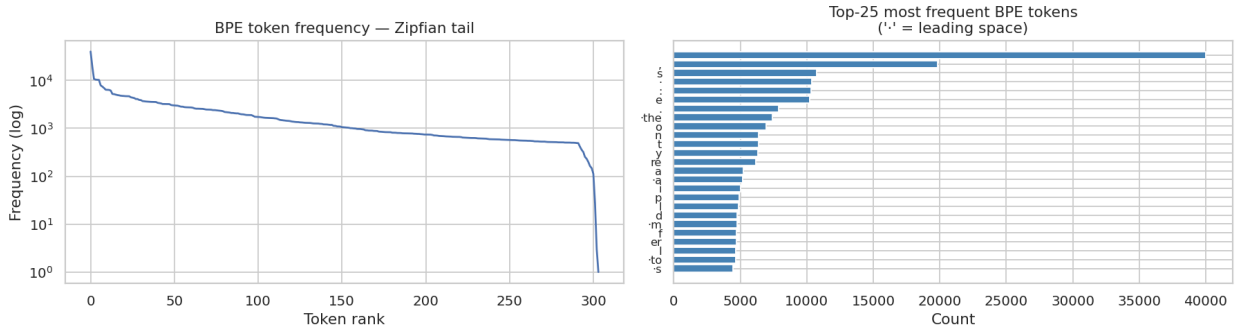


Figure 1: BPE token frequencies. Left: token count by rank on a log scale. Right: the 25 most frequent tokens. The symbol "·" marks a leading space.

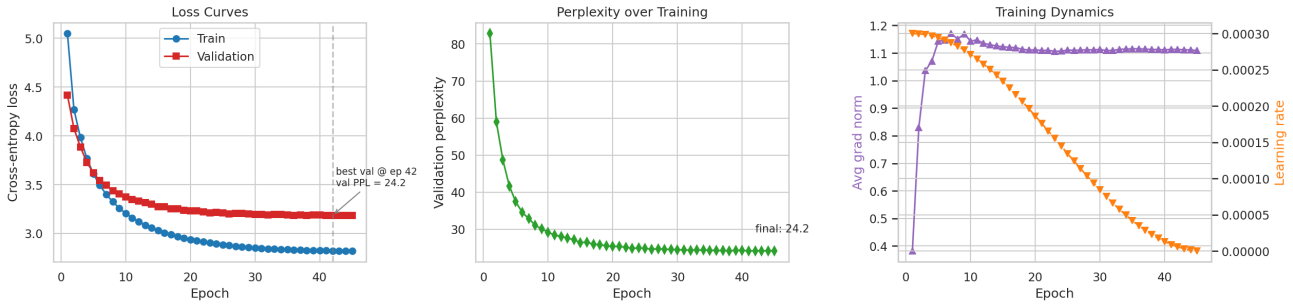


Figure 2: Training dynamics over 45 epochs. Left: training and validation cross-entropy loss. Middle: validation perplexity. Right: gradient norm and learning rate.

- residual connections around both the attention and feed-forward sublayers,
- RMSNorm before attention and before the feed-forward layer,
- final RMSNorm before the output projection,
- tied input and output embeddings.

I used sinusoidal positional encoding because it is simple, standard, and easy to reason about. Since the model only sees sequences of length 50 during training, I did not need anything more elaborate. I also used RMSNorm instead of LayerNorm because it is slightly simpler and was explicitly suggested by the assignment. At this scale I did not expect a dramatic difference, but I wanted the implementation to match the design I was aiming for. The most important piece in the whole model is the causal mask inside self-attention. Before applying the softmax, I set all attention scores above the diagonal to $-\infty$. After the softmax, those positions get zero probability mass, so each token can only attend to itself and earlier tokens. Without that mask, the model would be able to peek at future targets during training, which would completely break the next-token prediction setup. In every attention plot I made, all of the mass stayed on or below the diagonal, which is exactly what I wanted to see. The model has 327,552 parameters in total, or about 0.33 million. Most of them live in the embedding matrix and the feed-forward layers, not in the attention projections themselves. That is a useful reminder that even though attention gets most of the conceptual focus, the feed-forward part is still doing a large share of the numerical work.

4 Training

I trained the model for 45 epochs using AdamW with learning rate 3×10^{-4}, weight decay 0.01, cosine learning-rate decay, and gradient clipping at 1.0. The loss is cross-entropy averaged across all token positions in the sequence, not just the last one. That matters because under the causal mask every position is a valid next-token prediction problem. I also fixed the random seeds for Python, NumPy, and PyTorch so that the run is reproducible. This made it easier to compare results while tuning and also made the notebook much easier to debug.

Figure 2 summarizes the whole training run. The training loss drops from 5.05 to 2.82, and the validation loss drops from 4.42 to 3.19. The final validation perplexity is 24.18, with the best value of 24.16 reached at epoch 42. Since the vocabulary size is 500, this is far better than random guessing and shows that even a very small model can learn useful structure from this dataset. The shape of the curves is also informative. The loss falls quickly in the first few epochs, then continues to improve more gradually. The gap between the training and validation curves stays

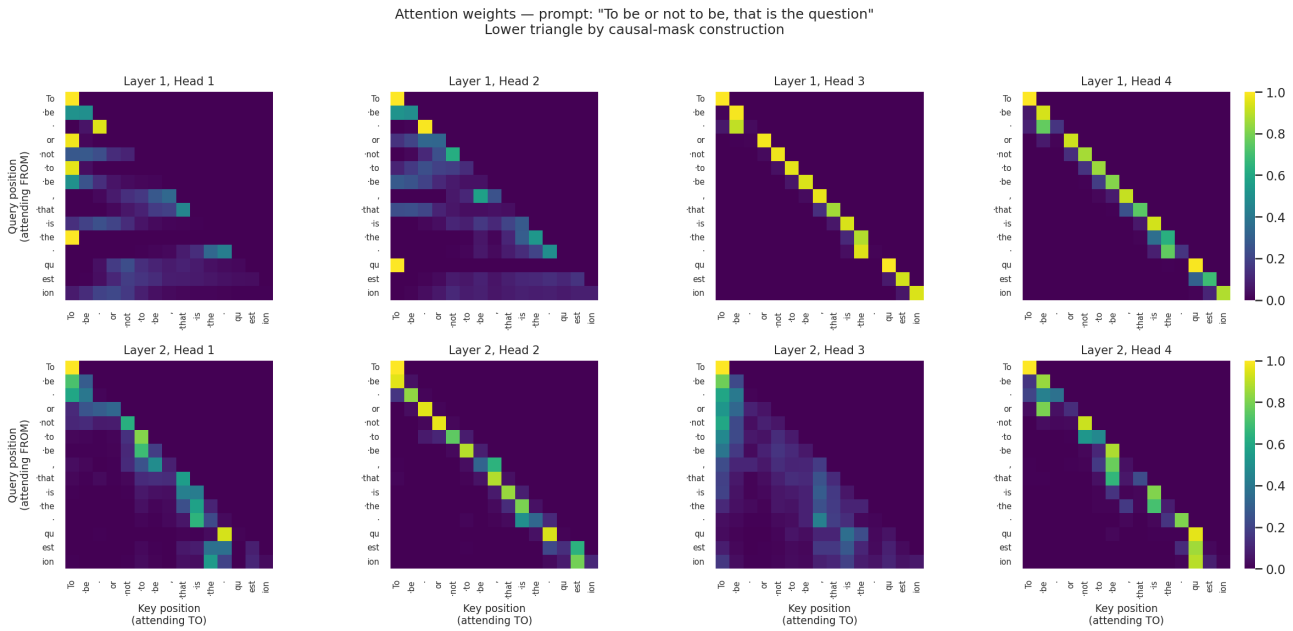


Figure 3: Attention weights for all eight heads (2 layers × 4 heads) on the fixed prompt. Cell (i, j) is the weight that query position i assigns to key position j.

fairly small throughout the run, which suggests that the model is not badly overfitting. If anything, the model looks a little underpowered for the full complexity of the corpus, which is completely fine for this assignment because interpretability mattered more to me than squeezing out the last bit of performance. The gradient norm plot adds another layer to the story. The first couple of epochs have the biggest movement, which makes sense because the model is still far from a useful solution. After that, the norm settles down and the training run becomes much steadier. Gradient clipping was mostly a safety measure. Early on it helped prevent large updates, but later in training it did not need to intervene much.

5 Attention Analysis

This was the most interesting part of the assignment for me. Once the model had finished training, I looked at the learned attention weights using the fixed prompt `To be or not to be, that is the question`. I chose this prompt because the tokens are readable and the sequence is short enough that the heatmaps remain easy to interpret.

5.1 Per-Head Patterns at the End of Training

Figure 3 shows the attention maps for all eight heads. Looking across the heads, I see three clear behavior types.

First, some heads are strongly diagonal. Layer 1 Heads 3 and 4, and Layer 2 Head 2, put most of their mass on the current token or one of the immediately previous tokens. These heads look like local information-preserving heads. They seem to be carrying forward short-range context without trying to summarize the whole prefix. Second, some heads put a surprising amount of attention on the first token in the sequence. Layer 1 Head 1 and Layer 2 Heads 1 and 4 have that pattern. I think of these as first-token anchor heads. No matter where the query is in the sequence, those heads often fall back to the start. In a small model, that looks like a simple but effective default behavior: when the head does not find a strong local signal, it uses a stable reference point. Third, Layer 2 Head 3 is much more diffuse than the rest. Instead of locking onto one position, it spreads attention across many earlier tokens. That makes it look like a rough averaging head that collects a broader summary of the context. I like this head because it is a good example of why multi-head attention is useful. Not every head needs to do the same job. Some can be sharp and local, while others can be broad and summarizing.

5.2 How Attention Changes During Training

To see how these patterns formed, I saved attention snapshots during training instead of only looking at the final model. Figure 4 shows Layer 1 attention for the same validation sample at epochs 1, 22, and 45. What stands out most is that the heads already have some structure very early. Even at epoch 1 they are not random clouds. That makes sense in hindsight: one full epoch already contains a large number of parameter updates for a model this small. By epoch 22 the patterns are much clearer and by epoch 45 they are noticeably sharper. An especially nice

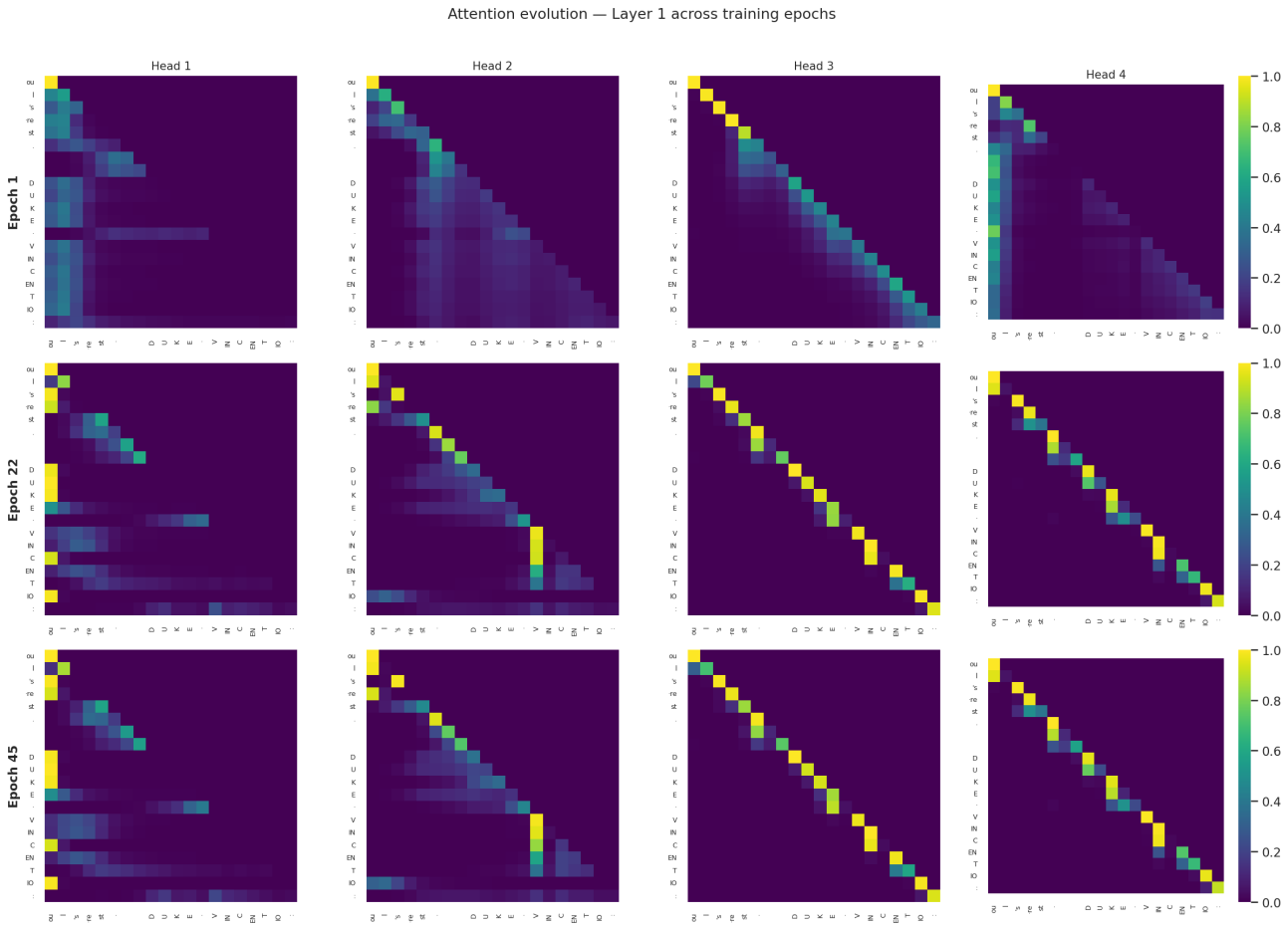


Figure 4: Layer-1 attention on the same validation sample at epochs 1, 22, and 45.

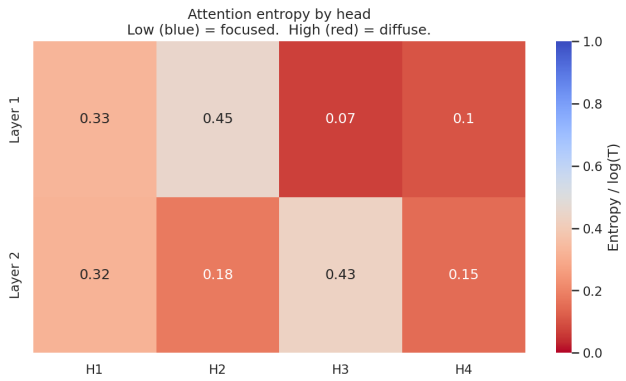
example is Head 2. At epoch 22 it still attends somewhat broadly, but by epoch 45 it focuses more strongly on the token "V" at the start of `VINCENTIO`. That feels meaningful rather than accidental. Once the model has recognized the start of a speaker name, it can use that information to help predict what comes next. More broadly, training snapshots suggest that heads do not invent brand-new roles late in training. Instead, they develop rough tendencies early and refine them over time.

5.3 Sample Generations

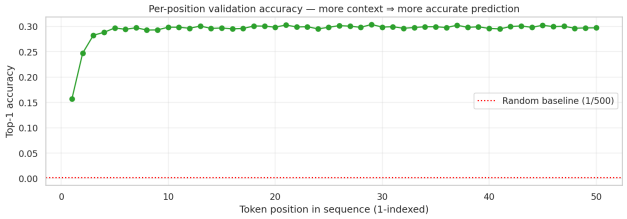
To get a qualitative feel for what the model had actually learned, I ran two short generation experiments using temperature 0.9 and top-k sampling with $k = 40$. The first prompt was `ROMEO:` and the second was `To be, or not to be`.

The outputs had a few things in common. The model consistently produced recognizable character names in all-caps followed by colons, which is the speaker-label format used throughout the Shakespeare corpus. It also tended to produce short declarative lines, which matches the rhythm of the training text. The content, however, fell apart quickly. Lines like "I did be gone out the wal of his villain" have the right surface shape but are grammatically broken, and invented words like "excoldice" appear within the first few lines. The second prompt produced a slightly more coherent opening, the model continued with something thematically related, but it also lost the thread within two or three sentences.

This is roughly what I expected from a 0.33M-parameter model trained on a small corpus. The model has clearly picked up document structure (who speaks, how turns are laid out) and some very common phrase fragments, but it does not have enough capacity to track meaning or enforce grammatical agreement over more than a few tokens. That gap between structural fluency and semantic coherence is a useful thing to see directly, because it makes clear what extra capacity actually buys in a larger model.

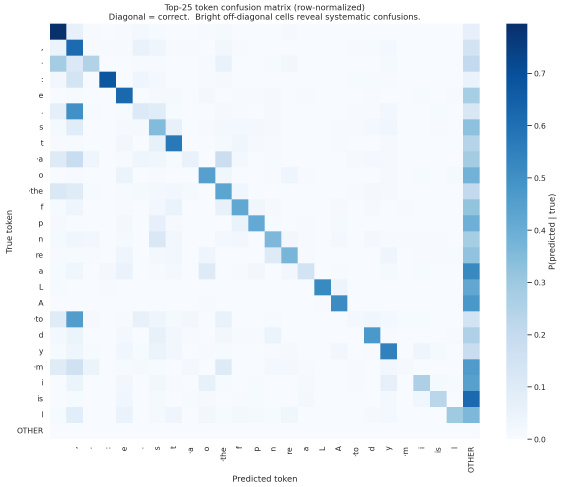


(a) Average attention entropy by head.

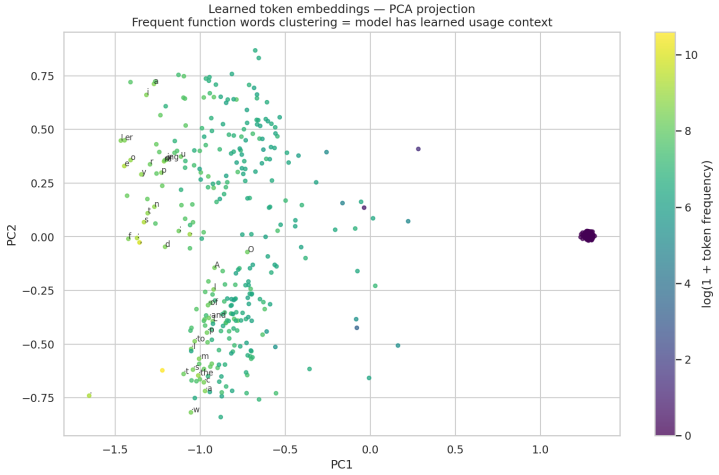


(b) Top-1 validation accuracy by token position.

Figure 5: Two diagnostics that quantify attention behavior and prediction quality.



(a) Confusion matrix for the top-25 targets.



(b) PCA projection of the learned token embeddings.

Figure 6: Two more diagnostics that helped me interpret what the model learned.

6 Additional Diagnostics

The attention heatmaps already told most of the story, but I added four more diagnostics to make the analysis less dependent on visual impressions alone.

6.1 Head Entropy

The first extra diagnostic was attention entropy. I computed the entropy of each head's attention distribution and normalized it by $\log T$, so that a value near 0 means the head is very focused and a value near 1 would mean it is close to uniform. Figure 5a matches what I saw by eye in the heatmaps. Layer 1 Head 3 is extremely focused, while Layer 2 Head 3 is much more diffuse. That gives a numerical version of the same story: some heads are pointer-like, while others are spread out over a larger part of the context.

6.2 Per-Position Prediction Accuracy

The second extra diagnostic was per-position top-1 accuracy, shown in Figure 5b. This plot helped me answer a simple question: how much context is the model really using? The first position has no left context at all, so the model can only rely on general token frequencies there. After that, accuracy rises quickly over the next few positions and then levels off near 30%. I take two things from that curve. First, the model is clearly using context, because a context-free predictor would stay much flatter. Second, at this size, most of the useful signal seems to come from only a handful of nearby tokens. A larger model trained on more data would probably keep benefiting over a longer range, but this small one gets most of its gain early.

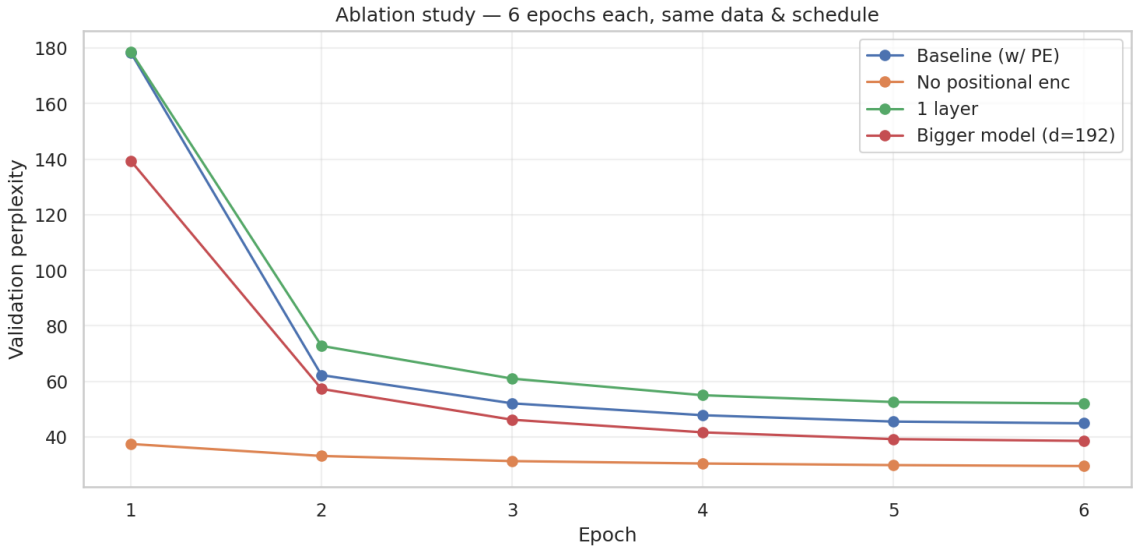


Figure 7: Validation perplexity over 6 epochs for four model variants.

6.3 Confusion Matrix

I also built a confusion matrix for the 25 most frequent target tokens, shown in Figure 6a. A full 500×500 confusion matrix would have been too hard to read, so I restricted the analysis to the most common tokens and grouped everything else into an `OTHER` bucket. The diagonal is clearly visible, which is a good sign, but there are also a few interesting mistakes. Some rows put a lot of mass into `OTHER`, which means the model often prefers a token outside the top 25. That is not automatically wrong, since the correct continuation is often a rarer token. I also noticed that "·to" is sometimes confused with a comma. That sounds odd at first, but both can appear in high-frequency grammatical contexts, so the mistake is not as random as it looks.

6.4 Embedding PCA

Finally, I projected the learned token embedding matrix down to two dimensions with PCA, shown in Figure 6b. Even though PCA is only a coarse view of the full 128-dimensional space, it still reveals structure. I can see one region with letters and suffix-like pieces, another with common function words and capital letters, and a separate cluster of very rare tokens whose embeddings barely moved during training. What I like about this plot is that the model learned that structure without any explicit supervision beyond next-token prediction. It was never told which tokens are function words or suffixes. That organization emerged from repeated contextual use.

7 Ablation Experiments

I ran four shorter training runs of 6 epochs each to compare a few design choices: the baseline model, a model without positional encoding, a one-layer model, and a wider model with $d = 192$.

Figure 7 contains the result that surprised me the most. The model without positional encoding finishes with lower perplexity than the full baseline after only 6 epochs. My first reaction was that I had probably made a mistake somewhere. After checking the implementation, I do not think that is what happened.

My interpretation is simpler. Without positional encoding, the model can lean heavily on easy frequency-based patterns and learn them quickly. On a short window and a small corpus, that turns out to be a strong early baseline. We fixed context length at 50 tokens; extending this would require more data and longer training but would likely improve long-range coherence. Once positional encoding is added, the model has the capacity to learn richer order-sensitive behavior, but it also takes longer to get there. That is consistent with the full 45-epoch baseline run, where the model reaches validation perplexity 24.18, clearly better than the short no-position run. The other ablations behaved more like I expected. The one-layer model plateaued earlier and at a worse perplexity, which suggests that a single block does not give the model enough depth to compose useful features. The wider $d = 192$ model did a little better than the baseline at the same 6-epoch budget, which points in the expected direction, though not by a huge margin. Overall, this section reminded me to be careful with short ablations. They are useful, but they can easily favor the model that learns the easiest patterns fastest rather than the model that ends up strongest in the long run.

8 Discussion

The main thing I learned from this assignment is that attention heads really do specialize, even in a tiny model. I went into the project expecting the heads to look noisy or redundant, but that is not what I found. Some heads became strongly local, some kept returning to the first token, and one head behaved like a broad context summary. The training snapshots made this even more convincing because the same roles showed up early and then sharpened over time. The hyperparameter that mattered most for stability was the learning rate. When I briefly tried 10^{-3}, the loss curve was much less smooth in the early epochs. The smaller learning rate of 3×10^{-4}, together with cosine decay, gave a much more stable run. Gradient clipping helped as a guardrail, but it was not the main reason training worked. The positional encoding result was also a useful lesson. If I had only looked at the 6-epoch ablation, I might have walked away with the wrong conclusion. The short run made it look as if positional information was hurting performance, but the longer run showed the opposite. What changed was not the importance of position itself, but the amount of time the model had to learn how to use it.

On the runtime and memory side, the full 45-epoch run finished in 7.1 minutes on a GPU, which works out to about 9.5 seconds per epoch. With a batch size of 64 and sequence length 50, each epoch processes roughly 910 batches of 64 sequences. The model itself takes up about 1.3 MB in memory (327,552 parameters at 32-bit float). The attention buffers add a modest overhead: at each layer the attention matrix has shape $64 \times 4 \times 50 \times 50$, which is about 2.5 MB per layer and 5 MB across both layers. That is negligible at $T = 50$, but it also makes the scaling problem concrete. If the sequence length were 10 times longer, say, $T = 500$, those same buffers would grow to 250 MB per layer, and at $T = 2048$ the attention alone would exceed 4 GB per layer at this batch size. That is the direct reason why long-context models need sparse or linear attention mechanisms rather than standard dense self-attention. For this assignment, the bottleneck was not memory but the forward and backward passes through the feed-forward layers, which dominate compute time at this scale.

9 AI Tool Usage Disclosure

Tools used: I used Claude for a few narrowly scoped tasks: sketching the early structure of the data pipeline, checking my causal-mask code for off-by-one mistakes, and helping me tighten some figure captions. I also used ChatGPT occasionally while debugging runtime errors, mostly when I had tensor-shape mismatches during development.

My own contribution: The model design, hyperparameter choices, training setup, plots, attention interpretation, ablation analysis, and all of the conclusions in this report are my own work.